Name: Nupoor Deshpande
PRN: 202401100026
Div: A Batch A2

Practical 2: Design, develop and deploy a web application to demonstrate server-side validation using ExpressJS.

Deployed Link: express-assignment
Github Repository for code: Codes

Aim: To design, develop, and deploy a modern Single Page Application (SPA) utilizing ExpressJS, express-validator middleware, and the Shadcn UI design system to demonstrate comprehensive server-side input validation, defensive error handling, data sanitization, and real-time request-response diagnostics.

Objectives:

- To implement server-side validation logic using ExpressJS and express-validator to ensure all incoming data conforms to strict structural, typing, and semantic constraints.
- To apply defensive sanitization principles including trimming whitespace, normalizing emails, and escaping HTML entities to neutralize Cross-Site Scripting (XSS).
- To utilize Shadcn UI component primitives (Cards, Forms, Inputs, Badges, Tabs, Alerts, and Toasts) with accessible aria-invalid error bindings for responsive design.
- To configure modular RESTful routes with precise HTTP status code semantics (200 OK, 201 Created, 400 Bad Request, 422 Unprocessable Entity).
- To build a real-time HTTP diagnostic console and audit log displaying live client-server execution details.
- To test the application thoroughly using valid, invalid, boundary, and malicious input scenarios.

Theory

1. Server-Side Validation Architecture & Defense-in-Depth:

Client-side validation provides immediate user feedback and improves ergonomics, but it runs on an untrusted environment that can be bypassed using tools like cURL, Postman, or disabled browser scripts. Consequently, **server-side validation serves as the authoritative security boundary**. Under OWASP guidelines, every payload crossing the network must undergo validation and sanitization prior to database persistence.

2. ExpressJS Middleware Pipeline & Request Handling Lifecycle:

ExpressJS handles requests through an ordered pipeline of middleware functions. Payloads are parsed via `express.json()` into `req.body`, after which declarative validation chains evaluate each field sequentially:

- **Sequential Middleware Execution:** Validation chains run as route-level middleware prior to controller handlers.
- **Error Interception via `validationResult(req)`:** Aggregates validation failures into a unified diagnostics set, terminating early with HTTP 422 Unprocessable Entity if errors exist.
- **Standardized Diagnostics:** Returns clean JSON structures containing field names, values, and targeted correction advice.

3. Input Sanitization, Cross-Field Rules & Status Codes:

- **Sanitization:** `trim()` eliminates boundary whitespace, `normalizeEmail()` standardizes email formats, and `escape()` converts characters (<, >, &, ") into HTML entities to mitigate XSS attacks.
- **Cross-Field & Semantic Logic:** Custom validation checks match password confirmation against password, verify that calculated age from Date of Birth is ≥ 18, and query database uniqueness for registered users.
- **HTTP Status Semantics:** Emits HTTP 201 on created entities, HTTP 422 on validation failures, and HTTP 400 on malformed JSON bodies.

Express Features & Shadcn UI Components Used:

Express / Shadcn Component	Purpose & Implementation in Server Validation Portal
Express Router `express.Router()`	Modular routing for registration, login, service tickets, and audit history endpoints.
Validation Chains `body().isLength().isEmail()`	Declarative validation rules applied as sequential middleware on request parameters.
Error Collector `validationResult(req)`	Extracts validation failures and returns formatted HTTP 422 diagnostic JSON payloads.
Sanitization Primitives `trim(), escape(), normalizeEmail()`	Strips whitespace, normalizes casing, and escapes HTML characters to neutralize XSS vectors.
Shadcn Field Primitives `FieldGroup, Field, FieldError`	Accessible form layout engine managing `data-invalid` and field-level error messages.
Semantic Form Controls `Input, Select, Textarea`	Styled form inputs with accessible `aria-invalid` bindings and focus states.
Alert Callouts `Alert, AlertTitle, AlertDescription`	Contextual alerts displaying server responses: green HTTP 201 on success and red HTTP 422 on validation errors.
Tabs Navigation `Tabs, TabsList, TabsTrigger`	Segmented switching between Registration, Authentication, Ticket, and JSON Sandbox views.
HTTP Diagnostic Console `Terminal, pre, code`	Live inspector displaying request payload, response status code, latency, and returned server JSON.

Procedure / Implementation Steps:

1. **Project Setup & Dependency Configuration:**
   - Create the modular fullstack project structure: `server/` for Express backend and `client/` for Vite React frontend.
   - Configure dependencies: `express`, `express-validator`, `cors`, `morgan`, Tailwind CSS, Lucide icons, and Shadcn UI primitives.
2. **Configuring Express Server & Middleware Pipeline:**
   - Implement `server/index.js` with `express.json()`, `express.urlencoded()`, CORS, and static client serving.
   - Add global error middleware catching malformed JSON payloads and returning clean HTTP 400 Bad Request responses.
3. **Defining Server-Side Validation Chains & Sanitizers:**
   - Create validation middleware in `server/middleware/validationRules.js` using express-validator.
   - Implement constraints for Full Name (alpha & spaces, 3-50 chars), Username (alphanumeric, no leading digits), and RFC Email format.
   - Enforce password complexity (min 8 chars, uppercase, lowercase, digit, special character).
   - Implement custom validators for password matching (`confirmPassword === password`) and chronological age verification (`age ≥ 18` from DOB).
4. **Building Controller Handlers & Audit Trail:**
   - Develop route controllers in `validationController.js` returning structured JSON entities upon validation success.
   - Implement an in-memory ring-buffer audit logger capturing every request timestamp, route, HTTP status code, and validation summary.
5. **Designing Accessible UI with Shadcn Component Primitives:**
   - Build form components using Shadcn patterns: `FieldGroup`, `Field`, `FieldLabel`, `Input`, `Select`, `Textarea`, `Alert`, and `Badge`.
   - Bind server-side validation error indicators directly to form inputs using `aria-invalid` and `data-invalid` tokens.
   - Incorporate quick-fill preset buttons enabling instantaneous loading of valid, weak-password, underage, and duplicate-email test fixtures.

1. **Connecting Asynchronous Client-Server Lifecycle & Diagnostics:**
    - Dispatch asynchronous Fetch API POST requests with JSON payloads to the Express API endpoints.
    - Build the Live HTTP Inspector displaying real-time target route, request payload, response status code, latency, and returned validation array.
2. **Automated Testing & Verification Suite:**
    - Develop automated test suite (`server/tests/validation.test.js`) using Node's native test runner to assert all 9 test scenarios.

Validation Test Matrix & Verification Results:

Test #	Endpoint	Test Input Scenario	HTTP Status	Actual Server Response & Behavior
TC-01	`POST /api/register`	All fields correctly filled, age ≥ 18, strong password	**201 Created**	Server validated all 8 constraints; returned sanitized user entity and active status.
TC-02	`POST /api/register`	Empty payload: `{}`	**422 Unprocessable**	Rejected; returned structured array with 6+ field diagnostics; highlighted inputs red.
TC-03	`POST /api/register`	Email: `not-an-email`, Password mismatch	**422 Unprocessable**	Rejected; flagged invalid email format and *"Passwords do not match"* error.
TC-04	`POST /api/register`	DOB: `2021-08-10` (Calculated age: 5 years)	**422 Unprocessable**	Rejected; *"Age requirement not met: You must be at least 18 years old"*.
TC-05	`POST /api/register`	Email: `admin@portal.dev` (Existing)	**422 Unprocessable**	Rejected; simulated database uniqueness check detected collision.
TC-06	`POST /api/login`	Valid identifier and 8+ char password	**200 OK**	Success; verified credentials and generated signed session token.
TC-07	`POST /api/login`	Missing password parameter	**422 Unprocessable**	Rejected; *"Password cannot be empty (min 6 characters)"* error emitted.
TC-08	`POST /api/feedback`	Rating: 10 (exceeds max 5), Comments < 20 chars	**422 Unprocessable**	Rejected; bounds validation caught rating and minimum length violation.
TC-09	`POST /api/register`	Malformed JSON: `{ "syntax": err,`	**400 Bad Request**	Caught by Express error middleware; returned JSON *"Malformed JSON payload"*.

Output Screenshot:

Figure 1: Express Server-Side Validation Portal Interface (Initial State & Minimal Shadcn Card Design)

Figure 2: Server-Side Validation Failure (HTTP 422 Unprocessable Entity - Underage <18 Field Diagnostics)

Figure 3: Server-Side Validation Success (HTTP 201 Created with Sanitized User Entity & Verified Audit Box)

Conclusion & Practical Verification:

The practical successfully demonstrates robust server-side input validation and defensive request handling in an ExpressJS environment. Using declarative `express-validator` middleware pipelines, all incoming user data undergoes strict schema validation, type enforcement, boundary checks, and sanitization before persistence. Defensive sanitization routines (including `trim()`, `normalizeEmail()`, and `escape()`) effectively neutralize Cross-Site Scripting (XSS) and injection vectors. Custom semantic rules enforce age requirements (≥ 18 years computed from Date of Birth) and password complexity. RFC-compliant HTTP status codes (`201 Created`, `422 Unprocessable Entity`, `400 Bad Request`, and `200 OK`) ensure transparent client-server communication. The application has been verified through automated unit tests and deployed live to Vercel.

Automated Test Suite Status	**✓ 9 Passed, 0 Failed (100% Assertion Success Rate)**
Live Production Deployment	https://express-assignment-virid.vercel.app/
GitHub Repository	https://github.com/debugonaut/express_assignment